SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 20%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

*I **D.VAISHNAVI(192524143)**certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering

1. **Reverse Engineering a Recommendation System**
   A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.
2. **Reverse Engineering a Smart Traffic Management System**
   A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.
3. **Reverse Engineering a Fraud Detection Pipeline**
   A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.
4. **Reverse Engineering a Personalized Learning System**
   An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.
5. **Reverse Engineering a Cloud-Based AI Monitoring System**
   A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

1. Reverse Engineering a Recommendation System

Objective

To analyze and reverse engineer a large-scale e-commerce recommendation system that uses real-time user and product data, PySpark RDDs, AI models, and intelligent agents to generate dynamic product recommendations.

Inferred System Architecture

User Activity → Data Collection → Distributed Storage → PySpark RDD Processing → Feature Extraction → Content-Based and Collaborative Recommendation Models → Recommendation Engine → Intelligent Agent → Personalized Products → User Feedback → Model Update.

Data Collection

The system collects product views, searches, clicks, purchases, ratings, cart activity, browsing history, product categories, prices, and timestamps. Real-time events are continuously added to the processing pipeline.

PySpark RDD Processing

Raw interaction data is converted into RDDs and distributed across cluster nodes. map() can extract user and product information, filter() can remove invalid events, groupByKey() can group activities by user, reduceByKey() can calculate product popularity, and join() can combine interaction data with product information.

Example RDD Operations

```
data = sc.textFile("user_activity.csv")
records = data.map(lambda x: x.split(","))
valid = records.filter(lambda x: x[2] != "")
user_products = valid.map(lambda x: (x[0], x[1]))
product_count = user_products.map(lambda x: (x[1], 1)).reduceByKey(lambda a, b: a + b)
```

Recommendation Processing

Content-based filtering recommends products similar to previously viewed or purchased products. Collaborative filtering identifies users with similar behavior and recommends products preferred by those users. A hybrid engine combines both results to improve accuracy.

Role of Intelligent Agents

A User Behavior Agent monitors recent actions, a Recommendation Agent updates the recommendation list, a Feedback Agent observes clicks and purchases, and a Learning Agent adjusts user preferences based on feedback. This allows recommendations to change dynamically.

Expected Result

The reverse-engineered system is a distributed recommendation pipeline capable of processing large-scale user-item interactions and continuously adapting recommendations based on real-time behavior.

2. Reverse Engineering a Smart Traffic Management System

Objective

To reverse engineer a city-wide intelligent traffic system that changes traffic signals in real time according to vehicle density, congestion, and traffic patterns.

Inferred System Architecture

Traffic Cameras + Road Sensors + GPS Data → Data Collection → Distributed Stream Processing → PySpark RDDs → Traffic Feature Extraction → Congestion Analysis → Intelligent Agents → Signal Control → Feedback → Continuous Adaptation.

Data Collection

The system receives vehicle counts, average speed, queue length, road occupancy, GPS positions, accident reports, and historical traffic patterns from cameras, sensors, GPS devices, and traffic control systems.

RDD-Based Processing

Incoming traffic events can be represented as RDD records containing road ID, timestamp, vehicle count, and speed. map() extracts traffic features, filter() removes incorrect readings, reduceByKey() aggregates vehicles by road, and groupByKey() groups traffic observations by time or location.

Example RDD Operations

```
traffic = sc.textFile("traffic_data.csv")
data = traffic.map(lambda x: x.split(","))
valid = data.filter(lambda x: int(x[2]) >= 0)
density = valid.map(lambda x: (x[0], int(x[2]))).reduceByKey(lambda a, b: a + b)
```

Decision Making Under Time Constraints

The system calculates congestion scores from vehicle density, speed, and queue length. If congestion becomes high, the signal-control agent can increase green time for the affected direction. Decisions must be produced quickly because traffic conditions change continuously.

Intelligent Agent Interactions

A Traffic Monitoring Agent collects conditions, a Congestion Agent estimates road congestion, a Signal Agent controls traffic lights, and an Emergency Agent gives priority to ambulances or other emergency vehicles. Agents share information before making decisions.

Expected Result

The reverse-engineered system provides adaptive traffic signal control, reduced congestion, and faster responses to changing traffic conditions.

3. Reverse Engineering a Fraud Detection Pipeline

Objective
To reconstruct a distributed banking fraud detection pipeline that identifies suspicious transactions within milliseconds using PySpark, anomaly detection, and intelligent agents.
Inferred System Architecture
Transaction Stream → Data Validation → PySpark RDD Processing → Feature Extraction → Distributed Anomaly Detection → Risk Scoring → Intelligent Agents → Fraud Alert / Approval → Feedback → Model Improvement.
Data Collection
The system collects transaction amount, customer ID, location, time, merchant, device information, payment method, transaction frequency, and previous transaction history.
Feature Extraction
Important features include transaction amount deviation, unusual location, transaction frequency, time-of-day behavior, rapid changes in spending, and device changes. These features are transformed into numerical values for anomaly detection.
RDD Processing
PySpark RDDs distribute transaction records across cluster nodes. map() extracts features, filter() removes invalid transactions, reduceByKey() calculates customer-level statistics, and join() combines transactions with customer profiles or historical data.
Example RDD Operations
```
transactions = sc.textFile("transactions.csv")
data = transactions.map(lambda x: x.split(","))
valid = data.filter(lambda x: float(x[2]) >= 0)
customer_txn = valid.map(lambda x: (x[1], float(x[2])))
```
Anomaly Detection
The system compares new transactions with normal customer behavior. A transaction can receive a higher anomaly score when its amount, location, time, or frequency differs significantly from the customer's normal pattern. High-risk transactions are sent for further verification.
Intelligent Agent Mechanism
A Monitoring Agent receives transactions, an Anomaly Agent evaluates suspicious behavior, a Risk Agent calculates the final risk level, and an Alert Agent immediately notifies the bank or blocks the transaction when necessary. A Learning Agent uses confirmed outcomes to improve future detection.
Expected Result
The reconstructed pipeline provides fast and scalable fraud detection while allowing intelligent agents to coordinate monitoring, risk analysis, alerts, and continuous learning.

4. Reverse Engineering a Personalized Learning System

Objective
To reverse engineer an online learning platform that automatically changes content difficulty according to student performance and engagement.
Inferred System Architecture
Student Activity → Data Collection → PySpark RDD Processing → Performance Feature Extraction → Learning Analytics → Student Model → Intelligent Agents → Adaptive Content → Feedback → Updated Learning Path.
Learner Data
The platform collects quiz scores, assignment marks, attendance, course progress, video watch time, number of attempts, incorrect answers, time spent on topics, and learning frequency.
RDD Transformations
map() can extract student and score information, filter() can remove invalid records, groupByKey() can group activities by student, reduceByKey() can calculate performance statistics, and join() can combine student activity with course and content information.
Example RDD Operations

```
students = sc.textFile("student_activity.csv")
data = students.map(lambda x: x.split(","))
valid = data.filter(lambda x: x[2] != "")
scores = valid.map(lambda x: (x[0], float(x[2])))
student_scores = scores.reduceByKey(lambda a, b: a + b)
```

Adaptive Learning

The system identifies whether a learner is performing at a high, medium, or low level. High-performing students can receive advanced content, average-performing students can receive regular practice, and low-performing students can receive simpler explanations and additional exercises.

Intelligent Agents

A Student Agent maintains the learner profile, an Assessment Agent analyzes performance, a Recommendation Agent selects learning resources, a Feedback Agent provides immediate guidance, and a Progress Agent tracks improvement.

Expected Result

The reverse-engineered platform supports personalized learning by continuously analyzing student behavior and dynamically adjusting content and feedback.

5. Reverse Engineering a Cloud-Based AI Monitoring System

Objective

To analyze a cloud platform that automatically detects system anomalies and scales computing resources using large-scale log processing, PySpark RDDs, and intelligent agents.

Inferred System Architecture

Cloud Logs + Metrics + Resource Data → Log Collection → Distributed Storage → PySpark RDD Processing → Feature Extraction → Anomaly Detection → Resource Prediction → Intelligent Agents → Auto Scaling / Alerting → Feedback.

Data Collection

The system collects application logs, CPU usage, memory usage, network traffic, request rates, response times, error counts, storage usage, and service health information.

PySpark RDD Processing

Large log files are converted into RDDs and partitioned across worker nodes. map() extracts log fields, filter() selects errors or abnormal events, reduceByKey() aggregates metrics, groupByKey() organizes logs by service, and join() combines logs with resource information.

Example RDD Operations

```
logs = sc.textFile("cloud_logs.txt")
records = logs.map(lambda x: x.split(","))
errors = records.filter(lambda x: x[2] == "ERROR")
error_count = errors.map(lambda x: (x[1], 1)).reduceByKey(lambda a, b: a + b)
```

Fault Tolerance

RDDs are resilient distributed datasets. Their lineage information allows lost partitions to be recomputed from previous transformations. This supports reliable processing even when a worker node fails.

Anomaly Detection and Scaling

The system monitors abnormal increases in CPU usage, memory consumption, request rates, response time, and errors. When demand increases, the scaling component can request additional resources. When demand decreases, unnecessary resources can be released.

Intelligent Agent Coordination

A Monitoring Agent observes system health, an Anomaly Agent identifies unusual behavior, a Scaling Agent adjusts cloud resources, an Alert Agent notifies administrators, and a Recovery Agent responds to service failures. Agents coordinate to maintain availability and performance.

Expected Result

The reverse-engineered system provides scalable cloud monitoring, fault-tolerant log processing, automatic resource scaling, rapid anomaly detection, and coordinated intelligent response.

Overall Comparison

System	Main Data	AI Function	Key Agents	Main Benefit
Recommendation	User-item activity	Personalized recommendation	User, Recommendation, Learning	Dynamic personalization
Traffic Management	Sensors, cameras, GPS	Congestion analysis	Traffic, Signal, Emergency	Real-time signal control
Fraud Detection	Financial transactions	Anomaly detection	Monitoring, Risk, Alert	Fast fraud prevention
Learning System	Student activity	Adaptive learning	Student, Assessment, Feedback	Personalized education
Cloud Monitoring	Logs and metrics	Anomaly detection & scaling	Monitoring, Scaling, Recovery	Reliable cloud operation

Common Reverse-Engineering Pattern

Data Sources → Distributed Data Collection → PySpark RDDs → RDD Transformations (map, filter, groupByKey, reduceByKey, join) → Feature Extraction → AI Model → Intelligent Agents → Decision/Action → Feedback → Continuous Adaptation

Conclusion: In all five systems, PySpark provides scalable distributed processing, RDD operations transform and aggregate large datasets, AI models perform prediction or anomaly detection, and intelligent agents coordinate autonomous decisions and adapt the system using feedback

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation